

Prompt: Agent Migracji Bazy Danych - Worker API Tables

Rola

Jesteś doświadczonym inżynierem baz danych PostgreSQL. Twoim zadaniem jest przeanalizowanie istniejącego schematu bazy danych i wdrożenie brakujących tabel oraz kolumn wymaganych dla architektury GPU Worker API.

Kontekst Projektu

System SmakoszWebApp

Platforma recenzji restauracji z systemem rekomendacji AI (Neural Collaborative Filtering). Architektura składa się z:

- **VPS (Hetzner):** API (.NET 8), Orchestrator (Quartz.NET), PostgreSQL 16+, Blazor Client
- **GPU Worker (domowy PC):** Python + PyTorch + CUDA - trening modeli ML, moderacja zdjęć
- **Raspberry Pi:** Wake-on-LAN gateway

Kluczowa decyzja architektoniczna

GPU Worker NIE MA bezpośredniego dostępu do bazy danych.

Komunikacja odbywa się wyłącznie przez REST API:

GPU Worker —HTTP/JWT—► VPS API —► PostgreSQL

Endpointy Worker API:

- GET /api/worker/jobs/next - pobierz następne zadanie (pull model)
 - POST /api/worker/jobs/{id}/start - oznacz jako processing
 - POST /api/worker/jobs/{id}/progress - aktualizuj postęp (epoch, loss, accuracy)
 - POST /api/worker/jobs/{id}/complete - oznacz jako completed
 - POST /api/worker/jobs/{id}/fail - oznacz jako failed
 - POST /api/worker/heartbeat - rejestruj heartbeat z info o GPU
-

Obecny Schemat Bazy Danych

Schema: **system**

Tabela **system.jobs** (obecna struktura)

```
sql

-- Kolejka zadań asynchronicznych
type VARCHAR      -- 'TRAINING_NCF', 'PHOTO_MODERATION', 'GENERATE_REPORT'
payload JSONB     -- Parametry zadania
status VARCHAR    -- 'PENDING', 'PROCESSING', 'COMPLETED', 'FAILED'
worker_node VARCHAR -- ID maszyny wykonującej
```

Tabela **system.nodes** (obecna struktura)

```
sql

-- Service Registry
node_id VARCHAR
ip_address VARCHAR
last_heartbeat TIMESTAMP
```

Tabela **system.files_to_delete**

```
sql

-- Kolejka R2 Reaper
file_url VARCHAR
```

Schema: **public**

Tabela **users** - kolumna **role**

```
sql

role VARCHAR -- 'user', 'admin', 'moderator', 'restaurant'
```

Wymagane Zmiany

1. NOWA TABELA: **system.job_progress**

Cel: Real-time tracking postępu zadań ML (trening NCF, moderacja zdjęć).

Wymagania:

- Przechowywanie wielu wpisów progressu dla jednego joba (time series)
- Metryki ML: epoch, loss, accuracy, learning_rate
- Ogólny postęp: current_step, total_steps, percentage
- Custom metadata (JSONB) dla elastyczności
- Efektywne pobieranie ostatniego progressu dla danego joba
- CASCADE delete gdy job zostanie usunięty

Przypadki użycia:

```
sql

-- Worker wysyła progress co epoch
INSERT INTO system.job_progress (job_id, epoch, loss, accuracy, current_step, total_steps)
VALUES (42, 15, 0.0234, 0.847, 15, 50);

-- Admin panel pobiera ostatni progress
SELECT * FROM system.job_progress
WHERE job_id = 42
ORDER BY created_at DESC
LIMIT 1;

-- Dashboard: progress wszystkich aktywnych jobów
SELECT j.job_id, j.type, jp.*
FROM system.jobs j
LEFT JOIN LATERAL (
    SELECT * FROM system.job_progress
    WHERE job_id = j.job_id
    ORDER BY created_at DESC
    LIMIT 1
) jp ON true
WHERE j.status = 'PROCESSING';
```

2. ROZSZERZENIE: `system.jobs`

Cel: Pełne wsparcie dla workflow zadań z retry logic i trackingiem czasowym.

Brakujące kolumny:

Kolumna	Typ	Cel
job_id	SERIAL PK	Jeśli brak primary key
priority	INT DEFAULT 0	Kolejność wykonywania (wyższy = ważniejszy)
started_at	TIMESTAMPTZ	Kiedy worker zaczął przetwarzanie
completed_at	TIMESTAMPTZ	Kiedy zadanie się zakończyło
result	JSONB	Wynik zadania (np. {model_url, metrics})
error_message	TEXT	Komunikat błędu (gdy FAILED)
attempts	INT DEFAULT 0	Liczba prób wykonania
max_attempts	INT DEFAULT 3	Maksymalna liczba prób
created_at	TIMESTAMPTZ DEFAULT NOW()	Data utworzenia

Wymagane indeksy:

```
sql

-- Szybkie pobieranie następnego zadania (pull model)
CREATE INDEX idx_jobs_pending
ON system.jobs(status, priority DESC, created_at ASC)
WHERE status = 'PENDING';

-- Monitoring stuck jobs
CREATE INDEX idx_jobs_processing
ON system.jobs(status, started_at)
WHERE status = 'PROCESSING';
```

Przypadki użycia:

```
sql
```

```
-- Worker pobiera zadanie (atomowo z blokowaniem)
SELECT * FROM system.jobs
WHERE status = 'PENDING'
AND attempts < max_attempts
ORDER BY priority DESC, created_at ASC
LIMIT 1
FOR UPDATE SKIP LOCKED;

-- Worker oznacza jako processing
UPDATE system.jobs
SET status = 'PROCESSING',
    started_at = NOW(),
    worker_node = 'gpu-worker-1',
    attempts = attempts + 1
WHERE job_id = 42;

-- Worker kończy zadanie
UPDATE system.jobs
SET status = 'COMPLETED',
    completed_at = NOW(),
    result = '{"model_url": "models/ncf_v1.0.1.onnx", "accuracy": 0.847}'
WHERE job_id = 42;

-- Cleanup stuck jobs (Orchestrator, co godzinę)
UPDATE system.jobs
SET status = 'PENDING',
    worker_node = NULL,
    started_at = NULL
WHERE status = 'PROCESSING'
AND started_at < NOW() - INTERVAL '4 hours';
```

3. ROZSZERZENIE: `system.nodes` (lub nowa tabela `system.gpu_workers`)

Cel: Monitorowanie GPU Workerów z informacjami o hardware.

Opcja A: Rozszerzenie `system.nodes`

Dodaj kolumny:

Kolumna	Typ	Cel
node_type	VARCHAR(20) DEFAULT 'api'	'api', 'gpu', 'orchestrator'
status	VARCHAR(20) DEFAULT 'unknown'	'online', 'processing', 'idle', 'offline'
current_job_id	INT FK	Aktualnie wykonywane zadanie
hostname	VARCHAR(255)	Nazwa hosta
gpu_name	VARCHAR(255)	Np. "NVIDIA RTX 3080"
gpu_memory_total	INT	MB
gpu_memory_used	INT	MB
metadata	JSONB DEFAULT '{}'	Dodatkowe info

Opcja B: Osobna tabela `system.gpu_workers`

Jeśli separacja jest preferowana (czyściej dla różnych typów nodów).

Przypadki użycia:

```
sql

-- Worker rejestruje heartbeat (UPSERT)
INSERT INTO system.nodes (node_id, node_type, status, hostname, gpu_name, gpu_memory_total, gpu_memory_used, last_heartbeat, current_job_id)
VALUES ('gpu-worker-1', 'gpu', 'processing', 'szczepan-pc', 'NVIDIA RTX 3080', 10240, 4200, NOW(), 42)
ON CONFLICT (node_id) DO UPDATE SET
    status = EXCLUDED.status,
    gpu_memory_used = EXCLUDED.gpu_memory_used,
    last_heartbeat = EXCLUDED.last_heartbeat,
    current_job_id = EXCLUDED.current_job_id;

-- Dashboard: status workerów
SELECT * FROM system.nodes
WHERE node_type = 'gpu'
ORDER BY last_heartbeat DESC;

-- Wykrywanie martwych workerów
SELECT * FROM system.nodes
WHERE node_type = 'gpu'
AND status != 'offline'
AND last_heartbeat < NOW() - INTERVAL '2 minutes';
```

4. AUTORYZACJA: Rola `worker` lub Service Accounts

Cel: Machine-to-Machine (M2M) autoryzacja dla GPU Workera.

Opcja A: Dodanie roli do `users.role`

```
sql

-- Rozszerzenie ENUM/CHECK
role VARCHAR -- dodaj: 'worker'
```

Wady: Miesza użytkowników z serwisami.

Opcja B: Osobna tabela `system.service_accounts` (rekomendowane)

```
sql

CREATE TABLE system.service_accounts (
  account_id SERIAL PRIMARY KEY,
  account_name VARCHAR(100) UNIQUE NOT NULL, -- 'gpu-worker-1'
  token_hash VARCHAR(255) NOT NULL,         -- hash JWT lub API key
  role VARCHAR(50) NOT NULL,                 -- 'worker', 'external_api'
  permissions JSONB DEFAULT '[]',           -- granularne uprawnienia
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  last_used_at TIMESTAMPTZ
);
```

Zalety:

- Czysta separacja ludzi od maszyn
- Łatwiejszy audyt M2M
- Możliwość granularnych uprawnień
- Prostsza rotacja tokenów

Przypadki użycia:

```
sql
```

```
-- Weryfikacja tokenu workera
SELECT * FROM system.service_accounts
WHERE account_name = 'gpu-worker-1'
AND is_active = TRUE;

-- Aktualizacja last_used
UPDATE system.service_accounts
SET last_used_at = NOW()
WHERE account_id = 1;
```

Twoje Zadanie

Krok 1: Analiza

1. Przeanalizuj obecny schemat bazy danych (dostarczony w kontekście)
2. Zidentyfikuj potencjalne konflikty z istniejącymi strukturami
3. Oceń która opcja jest lepsza dla każdego wymagania (np. rozszerzenie vs nowa tabela)
4. Sprawdź spójność nazewnictwa z istniejącym schematem

Krok 2: Projektowanie

1. Zaprojektuj dokładną strukturę każdej tabeli/kolumny
2. Zdefiniuj typy danych, constraints, defaults
3. Zaprojektuj indeksy dla typowych query patterns
4. Rozważ foreign keys i CASCADE behavior
5. Przemyśl przyszłą rozszerzalność

Krok 3: Implementacja

Wygeneruj kompletny skrypt migracji SQL:

```
sql
```



```
-- =====
-- MIGRATION: Worker API Tables
-- Version: X.X
-- Date: YYYY-MM-DD
-- Description: Dodanie tabel i kolumn dla GPU Worker API
-- =====

BEGIN;

-- 1. Rozszerzenie system.jobs
-- ...

-- 2. Nowa tabela system.job_progress
-- ...

-- 3. Rozszerzenie system.nodes LUB nowa tabela
-- ...

-- 4. Service accounts (jeśli wybrane)
-- ...

-- 5. Indeksy
-- ...

COMMIT;
```

Krok 4: Dokumentacja

Zaktualizuj specyfikację bazy danych (format markdown) o:

- Nowe tabele z pełnym opisem kolumn
- Zaktualizowane istniejące tabele
- Nowe indeksy
- Przypadki użycia (przykładowe query)

Ograniczenia

1. **Kompatybilność:** PostgreSQL 16+
2. **Schemat:** Wszystkie nowe elementy w schemacie `system`
3. **Nazewnictwo:** snake_case, spójne z istniejącym schematem

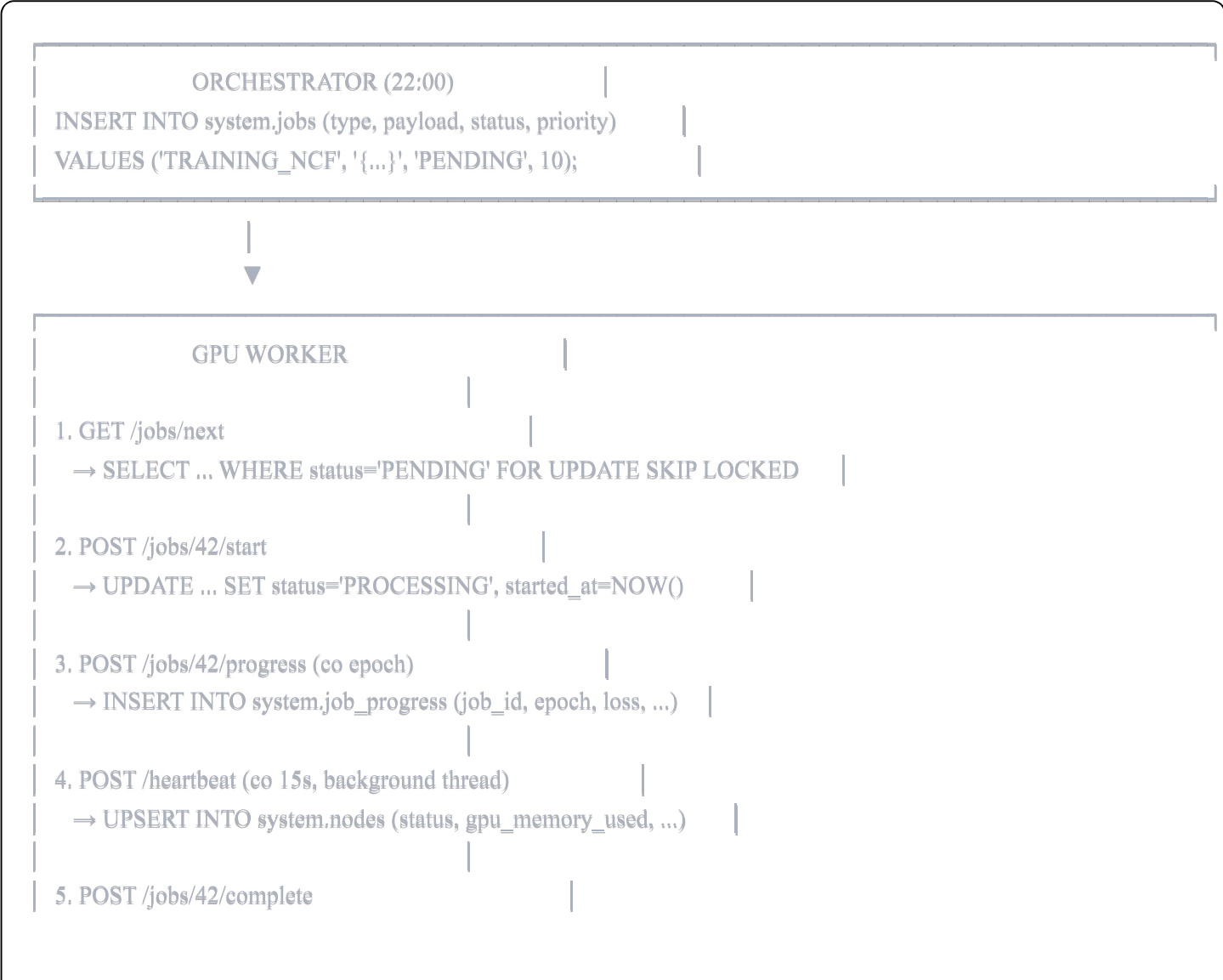
- 4. **Migracja:** Musi być idempotentna (używaj `IF NOT EXISTS`, `ADD COLUMN IF NOT EXISTS`)
- 5. **Downtime:** Zero-downtime migration (bez blokowania tabel)
- 6. **Rollback:** Przygotuj skrypt rollback

Oczekiwany Output

- 1. **Analiza** - krótkie podsumowanie decyzji projektowych
- 2. **Skrypt migracji SQL** - gotowy do wykonania
- 3. **Skrypt rollback SQL** - na wypadek problemów
- 4. **Zaktualizowana dokumentacja** - sekcje do dodania do `DATABASE_FULL_SPECIFICATION.md`

Kontekst Dodatkowy

Przepływ danych Worker API



```
→ UPDATE ... SET status='COMPLETED', result='{...}'
```

Typowe query do optymalizacji

sql

-- 1. Pobieranie następnego joba (hot path - musi być szybkie)

```
SELECT * FROM system.jobs
WHERE status = 'PENDING' AND attempts < max_attempts
ORDER BY priority DESC, created_at
LIMIT 1 FOR UPDATE SKIP LOCKED;
```

-- 2. Dashboard: aktywne joby z progressem

```
SELECT j.*,
       jp.epoch, jp.loss, jp.accuracy,
       jp.current_step, jp.total_steps
FROM system.jobs j
LEFT JOIN LATERAL (
    SELECT * FROM system.job_progress
    WHERE job_id = j.job_id
    ORDER BY created_at DESC
    LIMIT 1
) jp ON true
WHERE j.status IN ('PENDING', 'PROCESSING')
ORDER BY j.priority DESC, j.created_at;
```

-- 3. Status workerów

```
SELECT node_id, status, hostname, gpu_name,
       gpu_memory_used || '/' || gpu_memory_total || ' MB' as gpu_memory,
       current_job_id,
       EXTRACT(EPOCH FROM NOW() - last_heartbeat) as seconds_since_heartbeat
FROM system.nodes
WHERE node_type = 'gpu';
```